

TD n°4 : Les listes et l'arithmétique en Prolog

Choisissez votre éditeur préféré pour créer un fichier avec une extension `.pl` dans lequel vous écrirez votre programme, par exemple `tp4.pl`. Attention à ne pas utiliser de majuscule pour la première lettre du nom du fichier. Placez-vous dans le répertoire où figure votre fichier puis lancez Prolog grâce à la commande `pl`. Chargez votre programme grâce à l'instruction `[tp4]`. À chaque modification du programme, n'oubliez pas d'enregistrer et de recharger le fichier dans Prolog.

Si Prolog n'est pas installé dans votre machine, utilisez GNU Prolog sur la machine virtuelle Linux qui se trouve à l'adresse URL `m-vdi-etu-ha.univ-amu.fr`. Pour compiler un fichier `nom_du_fichier.pl`, utilisez la commande `gplc nom_du_fichier.pl` sur le terminal (dans le répertoire où figure votre fichier). Ça va créer l'exécutable `nom_du_fichier` que vous pouvez lancer en tapant `./nom_du_fichier` sur le terminal (dans le répertoire où figure votre fichier).

Préliminaires Les listes peuvent être définies de plusieurs façons différentes. La plus simple consiste à énumérer les éléments de la liste un par un. Par exemple : `[a, 42, f(3,a), [3, a]]` et la liste *vide* `[]`.

Comme vous pouvez le voir, une liste peut contenir des éléments de types différents, et elle peut contenir de tout : constantes, nombres, prédicats, listes, etc.

Une deuxième notation, plus structurée, permet de réaliser des inductions : on peut décrire une liste *non vide* comme `[X | L]` où `X` est la tête de la liste (premier élément) et `L` la queue de la liste (la liste, éventuellement vide, contenant tous les éléments sauf le premier). Par exemple, `[a,b,c]` est la même chose que `[a | [b | [c | []]]]`.

Exercice 1. Nous allons commencer par écrire des opérations de base sur les listes. Mettez à profit la structure inductive des listes, démontez les et remontez les comme il faut pour obtenir le résultat souhaité. On conseille fortement de sortir un stylo, une feuille de papier et de faire des schémas.

1. Définissez le prédicat `longueur/2` (c'est-à-dire que le prédicat `longueur` porte sur deux arguments) qui calcule la longueur d'une liste. Exemple :

```
?- longueur([1, 2, 3], A).  
A = 3.
```

Suggestion : Rappelez-vous que la *fonction* mathématique qui calcule la longueur d'une liste peut être définie de manière *récursive* comme il suit :

$$\begin{aligned}\text{long}([]) &= 0 \\ \text{long}([X | L]) &= 1 + \text{long}(L)\end{aligned}$$

Or, il faut utiliser le même "esprit récursif", en sachant que dans Prolog il n'y a pas de fonctions mais des prédicats. Donc, dans Prolog, `longueur(L,A)` veut dire que `A` est la longueur de la liste `L`.

2. Définissez le prédicat `somme/2` qui calcule la somme des éléments d'une liste. Exemple :

```
?- somme([1,2,3,4], X).  
X = 10.
```

3. Définissez le prédicat `membre/2` qui retourne `true` si un élément appartient à une liste une liste. Exemple :

```
?- membre(2, [1, 2, 3, 4]).
true.
```

```
?- membre(5, [1, 2, 3, 4]).
false.
```

4. Définissez le prédicat `ajoute_en_tete/3` qui ajoute un élément en tête de liste. Exemple :

```
?- ajoute_en_tete(1, [2, 3, 4], L).
L = [1, 2, 3, 4].
```

5. Définissez le prédicat `ajoute_en_queue/3` qui ajoute un élément en queue de liste. Exemple :

```
?- ajoute_en_queue(4, [1, 2, 3], L).
L = [1, 2, 3, 4].
```

6. Définissez le prédicat `extraire_tete/3` qui retourne la tête et la queue d'une liste. Exemple :

```
?- extraire_tete([1, 2, 3, 4], X, L).
X = 1,
L = [2, 3, 4].
```

7. Définissez le prédicat `concatene/3` qui concatène deux listes. Exemple :

```
?- concatene([1, 2, 3], [4, 5, 6], L).
L = [1, 2, 3, 4, 5, 6].
```

8. Définissez le prédicat `retourne/3` qui retourne les éléments d'une liste. Utilisez un accumulateur sous la forme d'un paramètre que vous initialiserez à `[]`. Exemple :

```
?- retourne([1, 2, 3], [], A).
A = [3, 2, 1].
```

Exercice 2.

1. Le tri par insertion utilise une fonction qui insère un élément à la bonne place dans une liste triée. L'algorithme principal se contente donc juste d'insérer les éléments un par un à l'aide de la fonction précédente.

- (a) Définissez le prédicat `insert_trie/3` qui insère un élément dans une liste triée à la bonne place. Exemple :

```
?- insert_trie(3, [1, 2, 4, 5], L).
L = [1, 2, 3, 4, 5].
```

- (b) Définissez le prédicat `tri_insert/2` qui trie une liste en utilisant le prédicat de la question précédente. Exemple :

```
?- tri_insert([5,2,4,3,1], L).
L = [1, 2, 3, 4, 5].
```

2. Le tri fusion utilise la stratégie « diviser pour régner » : la liste est divisée en deux parties égales, les deux parties sont triées récursivement, puis fusionnées en conservant l'ordre des éléments.

- (a) Définissez le prédicat `divise/3` qui divise une liste en deux parties égales (à un élément près). L'exemple suivant donne une idée de stratégie. Si vous songez à compter les éléments de la liste, ce n'est pas sérieux. Exemple :

```
?- divide([1,2,3,4,5], L, L2).
L = [1, 3, 5],
L2 = [2, 4].
```

- (b) Définissez le prédicat `fusion/3` qui fusionne deux listes, en conservant l'ordre des éléments. Exemple :

```
?- fusion([1,3,4], [2,5], L).
L = [1, 2, 3, 4, 5].
```

- (c) Définissez le prédicat `tri_fusion/2` qui trie une liste avec l'algorithme de tri fusion. Bien entendu, réutilisez le résultat des questions précédentes. Exemple :

```
?- tri_fusion([5,2,4,3,1], L).
L = [1, 2, 3, 4, 5].
```

3. Dans le tri rapide, on choisit un pivot dans la liste (par exemple le premier élément). Nous créons ensuite à partir de ce pivot et du reste de la liste deux nouvelles listes. La première contient les éléments plus petits que le pivot et la seconde ceux plus grands que le pivot. Il suffit ensuite de trier ces deux listes récursivement, et de concaténer le tout (sous-listes triées et pivot) pour obtenir la liste triée.

- (a) Définissez le prédicat `balance/4` qui étant donné un élément et une liste produit deux listes, l'une contenant les éléments plus grands et l'autre contenant les éléments plus petits. Exemple :

```
?- balance(3, [1, 2, 3, 4, 5], L1, L2).
L1 = [1, 2],
L2 = [3, 4, 5].
```

- (b) Utilisez le prédicat précédent pour définir le prédicat `tri_rapide/2` qui trie une liste à l'aide de l'algorithme de tri rapide. Vous pouvez utiliser le prédicat `concatene` défini précédemment. Exemple :

```
?- tri_rapide([5,2,4,3,1], L).
L = [1, 2, 3, 4, 5].
```

Exercice 3. Nous allons maintenant utiliser des listes pour coder des ensembles. Je vous rappelle que dans un ensemble, un élément ne peut appartenir au plus qu'une seule fois.

1. Définissez le prédicat `est_vide/1` qui retourne vrai si et seulement si l'ensemble passé en argument est vide. Exemple :

```
?- est_vide([]).
true.
```

```
?- est_vide([1]).
false.
```

2. Définissez le prédicat `ajoute_ensemble/3` qui ajoute un élément à un ensemble. L'élément n'est ajouté que s'il n'est pas déjà dans l'ensemble. Exemple :

```
?- ajoute_ensemble(2, [1, 2, 3], L).
L = [1, 2, 3].
```

```
?- ajoute_ensemble(4, [1, 2, 3], L).
L = [1, 2, 3, 4].
```

3. Définissez le prédicat `sous_ensemble/2` qui retourne vrai si et seulement si le premier ensemble est un sous-ensemble du second. Exemple :

```
?- sous_ensemble([4,2], [1,2,3,4]).
true.
```

```
?- sous_ensemble([4,2,5], [1,2,3,4]).
false.
```

4. Définissez le prédicat `union/3` qui retourne l'union de deux ensembles. Exemple :

```
?- union([1,3,2], [2,3,4], L).
L = [1, 2, 3, 4].
```

5. Définissez le prédicat `intersect/3` qui retourne l'intersection de deux ensembles. Exemple :

```
?- intersect([1,3,2], [2,3,4], L).
L = [3, 2].
```

```
?- intersect([1,5], [2,3,4], L).
L = [].
```

6. Définissez le prédicat `diff/3` qui retourne la différence de deux ensembles. Exemple :

```
?- diff([1,3,2,5], [2,3,4], L).
L = [1, 5].
```

Arithmétique. Prolog connaît les entiers et les nombres flottants. Il n'y a pas de valeur maximale pour les entiers ! Les expressions arithmétiques sont construites à partir de nombres, de variables et d'opérateurs arithmétiques. L'évaluation d'une expression arithmétique se fait par l'utilisation de l'opérateur `is`, par exemple `X is 3 - 2` est vraie si et seulement si `X` est égale à 1.

Prolog connaît un certain nombre d'expressions arithmétiques, plus précisément :

- des *opérations habituelles* avec notation infixe : addition (+), soustraction (-), multiplication (*), division entière (/), division flottante (/), modulo (mod), puissance (^);
- des *fonctions mathématiques* prédéfinies : `abs(X)`, `log(X)`, `sqrt(X)`, `exp(X)`, `sign(X)`, `random(X)`, `sin(X)`, `cos(X)`, `tan(X)`, `min(X,Y)`, `max(X,Y)`, `pi`, etc.

Par exemple, les interrogations suivantes donnent ça :

```
?- X is 2^20, Y is exp(1)
X = 1048576,
Y = 2.718281828459045.
```

```
?- X is sin(pi/2), Y is cos(pi).
X = 1.0,
Y = -1.0.
```

```
?- Z is sqrt(100)+100.
Z = 110.
```

```
?- S1 is sign(20), S2 is sign(-12).
S1 = 1,
S2 = -1.
```

En soi, les expressions arithmétiques sont juste de termes (on peut les voir comme des arbres). Elle ne seront *évaluées* que quand on les utilise avec `is` ou avec les opérateurs de comparaison : `==` (égalité), `=\=` (différence), `<` (inférieur strict), `=<` (inférieur large), `>` (supérieur strict), `>=` (supérieur large).

Attention à ne pas confondre ces opérateurs d'*évaluation* avec les opérateurs de *comparaison de termes* (`==` pour l'égalité de termes, `\==` pour la différence de termes) et avec les opérateurs d'*unification de termes* (`=` pour l'unification de termes, `\=` pour la non unifiabilité de termes).

Exercice 4. Posez à Prolog les interrogations suivantes, en essayant de comprendre la raison pour chaque réponse donnée par Prolog.

1. `?- 5 = 3+2.`
2. `?- 5 == 3 + 2.`
3. `?- 5 =:= 3 + 2.`
4. `?- f(X)=f(x).`
5. `?- f(X)==f(x).`
6. `?- f(X)=:=f(x).`
7. `?- f(X)\=f(x).`
8. `?- f(X)\==f(x).`
9. `?- f(X)=\f(x).`
10. `?- X is 9 mod 4.`
11. `?- X = 9 mod 4.`
12. `?- X == 9 mod 4.`
13. `?- f(x) =:= X+3.`
14. `?- f(X) = X+3.`
15. `?- f(X) = f(X+3).`
16. `?- X = 1+1+1, Y = X.`
17. `?- X = 1+1+1, Y is X.`
18. `?- X is X+1.`
19. `?- X is 2, X is X+1.`
20. `?- X is 2, X1 is X+1.`